OGP1 – Reliability & Maintenance Analytics
Alarm, failure and event impact analysis for an oil & gas processing plant (OGP1)

This project uses plant historian data, run status, failure events and DCS alarms to answer four business questions:

1. **How reliable is each machine?** It calculates MTBF, MTTR and availability for each asset.
2. **Where does downtime come from?** It builds a failure-mode Pareto and classifies failures as progressive or sudden.
3. **Do alarms warn early enough?** It compares the first DCS alarm with the P-F interval and follows ISA-18.2 / EEMUA 191 alarm KPIs.
4. **Can sensor data predict failures?** It uses a normal-behaviour health index and a supervised classifier, validated on unseen 2025 data.

The result is a **maintenance action list** (strategy per failure mode, a watch list and alarm fixes) and a **Power BI dashboard**.

Key results

#	Finding
1	**94 %** of unplanned downtime came from **progressive failures**, which can be predicted.
2	DCS alarms warned a median of **~4 days** ahead. On unseen 2025 data, the models warned a median of **11 days** ahead and caught **9 of 10 failures**.
3	**17 of 24** failure modes suit condition-based maintenance, with inspection intervals of **3–80 days**. The other **7 sudden modes** need root-cause analysis.
4	Recommended actions: a health-index watch list for **K-101 / GT-101**, CBM for pump bearings and seals, rationalisation of the **top-10 bad-actor alarms**, and an RCA of **K-101 spurious trips**.

Weibull results:

- For the **pumps**, β > 1, which means wear-out. Condition-based replacement pays off.
- For the **compressor train**, β < 1, which means early-life or random failures (spurious trips, surge). Fixing root causes works better than shortening PM intervals.

Project structure

```
OGP1_project_A/
├── README.md
└── Projecto_1/
    ├── run_all.py                # Runs the full pipeline (steps 01 → 08)
    ├── requirements.txt          # Python packages
    ├── src/
    │   ├── 01_data_cleaning.py
    │   ├── 02_load_sql.py
    │   ├── 03_reliability_analysis.py
    │   ├── 04_alarm_analysis.py
    │   ├── 05_condition_monitoring_eda.py
    │   ├── 06_failure_prediction.py
    │   ├── 07_maintenance_action_list.py
    │   ├── 08_powerbi_export.py
    │   ├── config.py             # Paths, source files and asset master data
    │   └── style.py              # Shared chart style
    ├── sql/
    │   ├── 01_create_schema.sql  # Star schema
    │   └── 02_kpi_queries.sql    # 12 KPI queries
```

```
    ├── notebooks/
    │   └── Ogp1_project1_alarm_failure_events_impact.ipynb   # Walkthrough with results
    ├── data/
    │   ├── raw/                    # Source Excel files (never edited)
    │   └── processed/              # Cleaned data, model outputs, action lists
    ├── models/                     # Trained models (.joblib), created by step 6 - not stored on GitHub
    ├── reports/
    │   ├── figures/                # fig01 … fig15 (PNG)
    │   ├── sql_results/            # Results of the 12 KPI queries (CSV)
    │   ├── Maintenance_Action_List.xlsx
    │   └── OGP1_Reliability_KPI_Summary.xlsx
    └── powerbi/
        ├── data/                   # Tables exported for Power BI
        └── OGP1_Reliability_Dashboard.pbix
```

Assets analysed

Asset	Description	Criticality
K-101	Gas compressor	A
GT-101	Gas turbine (K-101 driver)	A
P-201A / P-201B	Export pumps A and B	B
V-301	HP separator	A
E-401	Gas cooler / heat exchanger	B
TK-501	Storage tank	C
FLARE	Flare system	A

MTBF, MTTR and availability are calculated for the four rotating machines (K-101, GT-101, P-201A, P-201B), which have a run-status tag.

Pipeline: what each step does

Step	Script	What it does	Main outputs
1	01_data_cleaning.py	Cleans PI historian data. It handles text written into numeric columns (I/O Timeout, Bad Input, Calc Failed…), spikes and frozen instruments, and two wrong tag descriptions in the legend. Problems are **flagged rather than deleted**, and an audit trail is kept. Spikes are flagged only when the machine state was stable, because sudden changes during	data/processed/, data_quality_summary.csv

Step	Script	What it does	Main outputs
		trips or start-ups are real process behaviour.	
2	02_load_sql.py	Loads a **SQLite star schema** (dim_asset, dim_tag, fact_sensor_hourly, fact_run_status, fact_event, fact_alarm) and runs 12 KPI queries.	SQLite database
3	03_reliability_analysis.py	Calculates MTBF, MTTR, availability, the downtime Pareto, Weibull fits and the gap between the P-F interval and the first DCS alarm.	fig01–fig05
4	04_alarm_analysis.py	Alarm management to **ISA-18.2 / EEMUA 191**: alarm rates, bad actors, standing and repeat alarms, and priority mix.	fig06–fig07
5	05_condition_monitoring_eda.py	Condition-monitoring EDA: degradation signatures, K-101 correlations and ambient-temperature effects.	fig08–fig10
6	06_failure_prediction.py	Failure prediction in two layers (see below). The slowest step (under a minute on a normal PC).	fig11–fig14, models/
7	07_maintenance_action_list.py	Builds the RCM strategy per failure mode (CBM interval = P-F / 2, or RCA for sudden failures), the current watch	fig15, action list

Step	Script	What it does	Main outputs
		list with the sensor to check, the 2025 alert log (true or false) and fixes for bad-actor alarms.	
8	08_powerbi_export.py	Exports the tables for Power BI and creates the Excel KPI workbook.	powerbi/data/

Failure prediction method (step 6)

- **Layer A – normal-behaviour model (unsupervised).** For each condition sensor, a model predicts what the sensor *should* read from the operating context (ambient temperature, flow, speed, suction pressure). It is trained only on **healthy 2023–2024 hours.** Residual z-scores are combined into a **health index.**
- **Layer B – supervised classifier.** Gradient boosting on the residual features. The target is "inside the P-F interval".
- **Honest validation.** The models are trained on 2023–2024 and tested on **2025,** which they never saw.

Lesson learned: the classifier works well for the **pumps,** because they are two identical machines with repeated failure modes. For the **compressor train,** which has few and very different failure modes, it overfits and is almost always "on". The label-free **health index** is the right tool there, and it names the sensor that triggered each alert.

How to run

1. Requirements

- Python 3.10 or later
- Packages:

```
cd Projecto_1
pip install -r requirements.txt
```

2. Source data

The three source files are in Projecto_1/data/raw/:

- Facility_equipment_data.xlsx – PI historian sensor data
- equipment_event_failure_event.xlsx – failure and event log
- Alarm_log.xlsx – DCS alarm log

The raw files are never edited: every cleaning step writes to data/processed/. The main outputs (figures, Excel reports, SQL query results and Power BI tables) are included, so you can explore the results, the notebook and the dashboard without running anything. Large intermediate files (hourly sensor data, hourly predictions the SQLite database of about 270 MB, and the trained models) are not stored on GitHub: python run_all.py rebuilds them in about a minute.

3. Run the full pipeline

From the Projecto_1 folder:

```
python run_all.py
```

Command	What it does
python run_all.py	Runs every step, 01 → 08
python run_all.py 3	Starts at step 3. The outputs of steps 1–2 must already exist.

Command	What it does
python src/04_alarm_analysis.py	Runs one step on its own

The run **stops at the first step that fails** and prints the command to resume from that step. The full run takes about 1–2 minutes.
All outputs are written to data/processed/, reports/, reports/figures/, powerbi/data/ and models/.

4. Notebook walkthrough

```
cd Projecto_1/notebooks
jupyter notebook Ogp1_project1_alarm_failure_events_impact.ipynb
```

Run the **Setup** cell first. The notebook runs the 8 scripts in order and shows the key tables and figures inline.

Power BI dashboard

File: OGP1_Reliability_Dashboard.pbix. Open it with **Power BI Desktop**. It reads the tables exported by step 8. If the data does not load, go to **Transform data → Data source settings** and point it to your Projecto_1/powerbi/data/ folder.

Data model: fact_asset_daily, fact_events, fact_alarms, fact_health_daily, dim_date, plus a _Measures table.

Every page has **Year** and **Asset** slicers.

Page	Content
Overview	KPI cards (MTBF, MTTR, Availability %, Unplanned downtime, Failures, % Downtime predictable). Availability and MTBF by asset, with a 97 % availability target line. Downtime Pareto by failure mode.
Alarms	Alarms per day, max alarms in a day, % standing, % repeat and % high-priority alarms. Monthly alarms by asset. Top-10 bad actors. Priority mix compared with the 80 / 15 / 5 % guide. Table of what to change in the DCS.
Predictive	Failures caught in 2025. Median model warning compared with median alarm warning (days). Lead time per event (alarm vs model vs P-F interval). Daily risk timeline with alert level. Which sensor triggers the 2025 alerts.
Actions	Current watch list with status, main driver sensor and recommended action. Maintenance strategy by failure mode. CBM inspection interval (P-F / 2). 2025 alert log with true and false alerts.

Main DAX measures

Measure	Definition
Availability %	(Calendar h − Maintenance h − Tripped h) / Calendar h
MTBF (h)	Running hours / Number of failures
MTTR (h)	Average downtime per failure
Unplanned Downtime (h)	Sum of downtime for unplanned events
% Downtime Predictable	Unplanned downtime from *progressive degradation* / Total unplanned downtime
Pareto Cumulative %	Cumulative share of downtime by failure mode
Median Model Warning (days) / Median Alarm Warning (days)	Median lead time before failure

---

Figures

Figure	Content
fig01_mtbf_availability	MTBF and availability per asset
fig02_downtime_pareto	Downtime Pareto by failure mode
fig03_event_timeline	Event timeline
fig04_weibull	Weibull analysis
fig05_warning_time_gap	P-F interval compared with the first DCS alarm
fig06_alarm_bad_actors	Top bad-actor alarms
fig07_alarm_trend	Alarm trend
fig08_degradation_signatures	Degradation signatures
fig09_k101_correlation	K-101 sensor correlation
fig10_ambient_effect	Ambient-temperature effect
fig11_risk_timeline_2025	2025 risk timeline
fig12_lead_time_model_vs_alarm	Model lead time compared with alarm lead time
fig13_feature_importance	Feature importance
fig14_normal_behaviour_example	Example of the normal-behaviour model
fig15_inspection_intervals	CBM inspection intervals

Tools and standards

Python (pandas, NumPy, Matplotlib, seaborn, SciPy, scikit-learn, joblib) · **SQL** (SQLite) · **Jupyter** · **Power BI** (DAX) · **Excel**

Reliability methods: MTBF/MTTR, Weibull analysis, P-F interval, RCM, CBM and RCA. Alarm management: **ISA-18.2** and **EEMUA 191.**

Glossary

Term	Meaning
MTBF	Mean Time Between Failures
MTTR	Mean Time To Repair
P-F interval	Time between the first detectable sign of a failure (P) and the functional failure (F)
CBM	Condition-Based Maintenance
RCM	Reliability-Centred Maintenance
RCA	Root-Cause Analysis
DCS	Distributed Control System
Bad actor	An alarm that fires far more often than it should
Standing alarm	An alarm that stays active for a long time
Health index	Combined score of how far sensors deviate from their expected normal behaviour